Weekly Synthetic Data Research Agent

1. The task in one page

You will build an AI agent that, every Saturday, finds the most important new articles, papers, posts and tools about **synthetic data** from the past 7 days, writes a short report with links, and emails it to 5 recipients (listed in section 6).

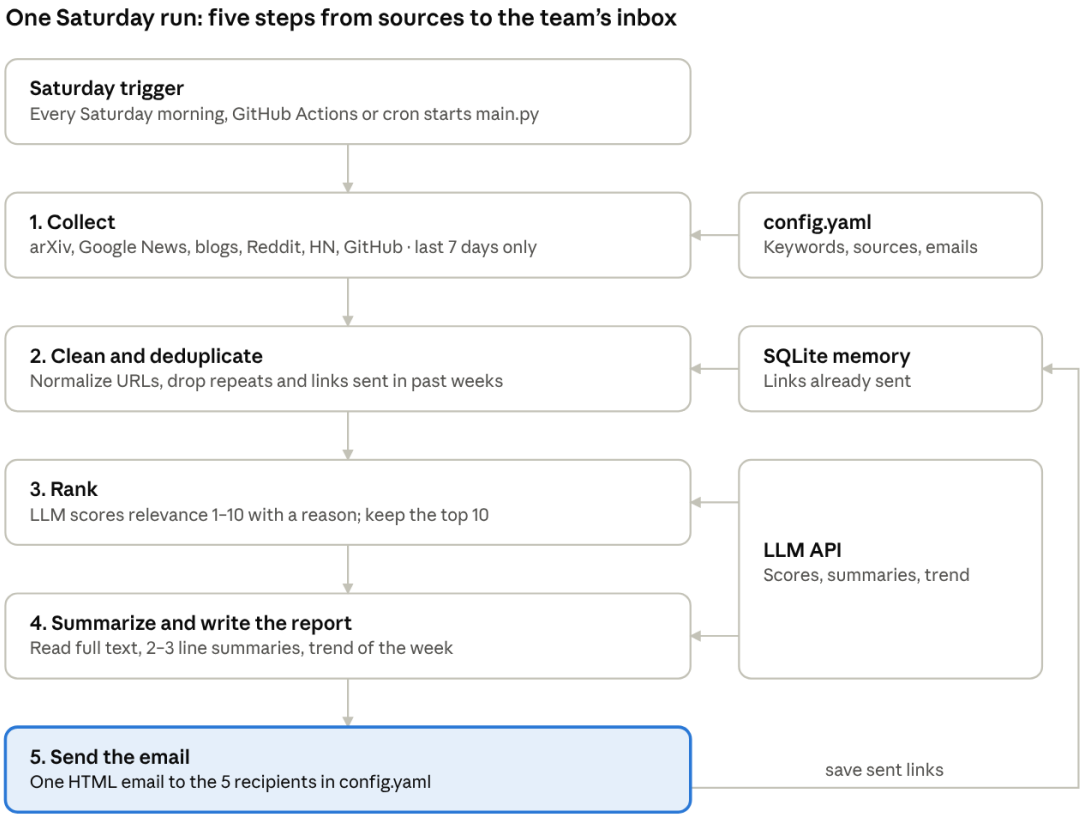
Think of it as a junior research analyst that never forgets to do its weekly homework. Your job is to design it, build it, test it, and make it run by itself.

Item	What it means
Topic	Synthetic data: generation methods, LLM-generated training data, tabular and privacy-preserving synthetic data, evaluation of synthetic data, new tools and datasets, industry news
Time window	Content published in the last 7 days only
Output	A brief report: 5 to 10 top items, each with a 2 to 3 line summary and its link
Recipients	5 recipients, listed in section 6 (kept in a config file, never hard-coded)
Schedule	Every Saturday morning, automatically
Done means	The agent runs unattended for 2 Saturdays in a row and the team finds the reports useful

The hard part is not fetching links. It is **choosing what matters**, removing duplicates and noise, and writing summaries that are short and correct.

2. How the agent works

The agent is a pipeline of 5 steps that runs once a week. In LangGraph, each box below becomes one node, and they all read and write one shared state.



agent pipeline · 5 steps

Read it top to bottom: the trigger starts the run, raw items are collected, cleaned, ranked and summarized, then the email goes out and the sent links are saved so they never repeat. The LLM is used only where judgment is needed (ranking and writing); everything else is plain Python.

3. Recommended tech stack

Everything is free except the LLM API; all sources are coded directly, with no paid search API.

Layer	Tool	Why
Language	Python 3.11+	Team standard, best AI library support
Agent framework	LangGraph	Each step becomes a node in a graph; easy to debug and extend
LLM	The LLM API the team already uses (e.g. Claude)	Scoring relevance, summarizing, writing the report
Research papers	arXiv API (`export.arxiv.org/api/query`,	Where most new synthetic data research appears first

Layer	Tool	Why
	sorted by submission date)	
News and articles	Google News RSS search (`news.google.com/rss/search?q="synthetic data" when:7d`)	Searches the open web for news; replaces a paid search API
Blogs	RSS feeds of research and company blogs, read with `feedparser`	Feed list kept in `config.yaml`
Community posts	Reddit RSS (r/MachineLearning, r/LocalLLaMA), Hacker News Algolia API	Discussions and new tools
New tools and repos	GitHub search API	New synthetic data libraries and datasets
Page reading	`httpx` + `trafilatura`	Downloads a page and extracts the main text
Memory / dedup	SQLite (one table of already-sent URLs)	Never sends the same link twice
Email	Gmail SMTP with an app password, HTML built with Jinja2	Formatted email with clickable links
Scheduling	GitHub Actions scheduled workflow, or cron on a team server	Runs every Saturday without anyone pressing a button
Config and secrets	`config.yaml` + `.env`	Keywords, sources and recipient emails in config; API keys in `.env`, never in Git

Do not scrape LinkedIn, X, or Google search result pages. They break terms of service, get blocked quickly, and fail without warning. The free sources above are official and stable.

4. Step-by-step build plan

Build it in 7 phases (0 to 6) over about 4 weeks. Finish and show each phase before starting the next; small working pieces beat one big unfinished agent.

1. **Phase 0 — Understand the topic (days 1–2)**
    - Read 3 to 5 introductions to synthetic data so you know what is important and what is noise.
    - Make one report **by hand**: spend 2 hours searching, pick the top 7 items, write the summaries. This is the gold standard your agent must match.

    - Write a list of 15–20 sources and 10–15 keywords (e.g. "synthetic data", "synthetic training data", "data augmentation LLM", "tabular data synthesis", "differential privacy synthetic").
    - Deliverable: the manual report + sources and keywords list.
2. **Phase 1 — Collectors (days 3–5)**
    - Write one small function per source: arXiv, Google News RSS, blog RSS feeds, Reddit RSS, Hacker News, GitHub.
    - Every collector returns the same shape, so the rest of the pipeline does not care where an item came from.
    - Keep only items published in the last 7 days.
    - Deliverable: a script that prints all raw items found this week.
3. **Phase 2 — Clean, filter and rank (week 2)**
    - Normalize URLs (remove tracking parameters) and drop duplicates, including links already sent in past weeks (SQLite).
    - Ask the LLM to score each item 1–10 for relevance to synthetic data and give a one-line reason. Keep the top 10.
    - Deliverable: a ranked list you can compare with your manual report.
4. **Phase 3 — Summarize and write the report (week 2)**
    - Fetch the full text of the top items and summarize each in 2–3 lines.
    - The prompt must say: summarize only from the given text, never invent facts, numbers or authors.
    - Add a 3–4 sentence "trend of the week" at the top.
    - Deliverable: a report in Markdown/HTML that follows the template in section 5.
5. **Phase 4 — Wire it as a LangGraph agent (week 3)**
    - Turn each step into a node; pass data through one shared state object.
    - If one source fails, log it and continue; the report must still go out.
    - Deliverable: one command (`python main.py`) runs the whole pipeline.
6. **Phase 5 — Email and schedule (week 3)**
    - Add a `--dry-run` flag that sends only to you. Use it for every test.
    - Set up the Saturday schedule (section 6).
    - Deliverable: you receive a correct test email.
7. **Phase 6 — Pilot and improve (week 4)**
    - Run for real for 2 Saturdays. Ask the 5 recipients for quick feedback and adjust keywords, sources and prompts.

A starting point for the shared data shapes:

```
from typing import TypedDict
from pydantic import BaseModel
from datetime import datetime

class Item(BaseModel):
    title: str
    url: str
    source: str          # "arxiv", "news", "blog", "reddit", "hn", "github"
    published: datetime
    snippet: str = ""
```

```
    full_text: str = ""
    score: int = 0
    summary: str = ""

class AgentState(TypedDict):
    raw_items: list[Item]
    ranked_items: list[Item]
    report_html: str
    errors: list[str]
```

5. Report format

The report must be readable in under 5 minutes. Every item has a title, a source, a date, a short summary and a working link.

```
Subject: Synthetic Data Weekly — Week of 18–24 Oct 2026

Trend of the week
3–4 sentences on the main theme across this week's items.

Top picks
1. [Title of article or paper](link)
   Source · Date published
   2–3 line summary: what it is, why it matters for our team.

2. [Title](link)
   ...

New tools and datasets
- [Tool name](link) — one line on what it does.

Worth a look (lower priority)
- [Title](link) — one line.

Auto-generated by the Synthetic Data Research Agent. Reply with feedback.
```

Limit: 5–10 top picks, 0–5 tools, 0–5 "worth a look" items. If fewer good items exist that week, send fewer; never pad the report.

6. Scheduling and email delivery

The agent runs every Saturday morning by itself and sends one email to all 5 recipients.

Recipients live in `config.yaml`, so adding or removing a person never needs a code change:

```
schedule: "Saturday 08:00"
recipients:
  - yasminemehrez890@gmail.com
  - lailabashmal@gmail.com
  - sbashmal@awqef.sa
  - mohammed.abdo@awqef.sa
  - umairafzal92786@gmail.com
max_top_picks: 10
lookback_days: 7
```

Sending: use a dedicated sender account (not your personal one) with an app password or the Gmail API. Store the password in `.env` or as a GitHub secret. Send one HTML email with all 5 recipients in the To field.

Option A — GitHub Actions :

```
# .github/workflows/weekly-report.yml
name: Weekly synthetic data report
on:
  schedule:
    - cron: "0 7 * * 6"   # Saturday 07:00 UTC — convert from your local time
  workflow_dispatch:       # lets you run it manually for testing
jobs:
  run:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: python main.py
        env:
          LLM_API_KEY: ${{ secrets.LLM_API_KEY }}
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}   # free, built in; for GitHub
search
          SMTP_PASSWORD: ${{ secrets.SMTP_PASSWORD }}
```

Note: GitHub cron is in UTC and can start a few minutes late. The SQLite file must be saved between runs (commit it, or use a small hosted database), or deduplication across weeks will not work.

Option B — cron on a team server: `0 8 * * 6 cd /path/to/agent && python main.py >> logs/run.log 2>&1`

7. Quality rules and common pitfalls

A report that is wrong once loses the team's trust. These rules matter more than any extra feature.

Quality rules

- Every link is opened and checked (HTTP 200) before it goes in the report.
- Every summary comes only from the item's real text. If the text could not be fetched, use the abstract or snippet and say so.
- The published date is checked; nothing older than 7 days enters "Top picks".
- No link appears in two different weeks.
- Each report keeps a log: how many items were found, filtered, and sent, plus any source errors.

Success criteria

- Runs every Saturday with no manual action
- At least 5 useful items in most weeks
- 0 broken links and 0 invented facts in the pilot reports
- Agent's top picks overlap clearly with your manual Phase 0 report

- Recipients say it is worth reading

Common pitfalls

Pitfall	How to avoid it
Too many generic "what is synthetic data" blog posts	Tell the ranking prompt to prefer new research, tools and real news over explainers
Same story from 5 websites	Group items with similar titles; keep the original source
LLM invents details	Strict prompt + summarize only fetched text + spot-check summaries yourself during the pilot
API costs grow	Only fetch full text and summarize for the top 10, not for everything
API keys pushed to GitHub	`.env` in `.gitignore` from day 1; use GitHub secrets
Spamming the team while testing	Always test with `--dry-run`

8. Check-ins and what to submit

When	
End of week 1	Manual report, sources and keywords list, collectors printing raw items
End of week 2	Ranked list vs. your manual report, first generated report
End of week 3	Full LangGraph pipeline, dry-run email received, schedule set up
End of week 4	2 real Saturday reports sent, feedback collected, list of improvements

Final submission

- Git repository with a clear `README.md`: what it does, how to set it up, how to run it, how to change recipients and keywords
- `config.yaml`, `requirements.txt`, `.env.example` (no real keys)
- The workflow or cron file
- A one-page note: what works, what doesn't yet, and 3 ideas for next steps (e.g. a Slack version, a searchable archive of past reports)